

RESUME MATERI PERKULIAHAN

Perancangan dan Analisis Algoritma (A)

Pertemuan ke-2

Disusun Oleh:

Gilbran Mahdavia Raja

NRP: 5025241134

Institut Teknologi Sepuluh Nopember

Surabaya

2026

Daftar Isi

1	Resume Materi Perkuliahan	2
1.1	Konsep Dasar: Algoritma, Program, dan Struktur Data	2
1.2	Pengelolaan Memori dan Arsitektur Komputer	2
1.3	Abstraksi: Menjembatani Dunia Nyata dan Dunia Komputasional	2
1.4	Prinsip <i>Worst Case Scenario</i>	3
2	Pembahasan Kasus: SPOJ CBANK — <i>Charu and Coin Distribution</i>	4
2.1	Deskripsi Permasalahan	4
2.2	Langkah 1: Memahami Logika Soal	4
2.3	Langkah 2: Abstraksi dan Penyederhanaan	4
2.4	Langkah 3: Menemukan Pola melalui Observasi	5
2.5	Langkah 4: Derivasi Model Matematika dengan <i>Newton's Forward Difference Formula</i>	5
2.6	Langkah 5: Penanganan Pembagian dalam Aritmetika Modular	6
2.7	Langkah 6: Implementasi — Solusi Optimal dengan Operasi Bitwise	6
3	Kesimpulan	8
4	Bukti <i>Accepted</i>	9

Resume Materi Perkuliahan

1.1 Konsep Dasar: Algoritma, Program, dan Struktur Data

Ilmu komputer bertumpu pada tiga konsep yang saling berkaitan erat:

1. **Algoritma:** Rangkaian instruksi langkah demi langkah yang terdefinisi dengan jelas untuk menyelesaikan suatu permasalahan tertentu. Algoritma bersifat abstrak dan belum terikat pada bahasa pemrograman mana pun. Agar valid, suatu algoritma harus memenuhi sifat-sifat: menerima *input*, menghasilkan *output*, bersifat *definite* (setiap langkah tidak ambigu), *finite* (pasti berhenti), dan *effective* (setiap langkah dapat dikerjakan).
2. **Program:** Wujud konkret dari algoritma yang ditulis dalam suatu bahasa pemrograman tertentu, seperti C, C++, Python, atau Java. Program adalah realisasi dari konsep algoritmik yang masih bersifat abstrak.
3. **Struktur Data:** Cara pengorganisasian dan pengelolaan data di dalam memori sehingga dapat diakses dan dimanipulasi secara efisien. Struktur data diperlukan karena memori komputer bersifat linear, sementara banyak masalah di dunia nyata memerlukan representasi yang tidak linear, seperti *tree*, *graph*, atau *heap*.

Hubungan antara ketiga konsep ini terangkum dalam pernyataan klasik Niklaus Wirth, pencipta bahasa Pascal:

$$\text{Program} = \text{Algoritma} + \text{Struktur Data}$$

1.2 Pengelolaan Memori dan Arsitektur Komputer

Setiap algoritma yang berjalan di komputer bergantung pada pengelolaan memori. Secara arsitektural, memori komputer tersusun secara linear data ditempatkan pada deretan alamat yang berurutan. Oleh karena itu, memahami karakteristik memori linear ini sangat penting dalam merancang struktur data yang tepat, sebab pola akses dan alokasi memori berdampak langsung pada efisiensi kinerja algoritma.

Sebagai contoh, struktur *stack* mengimplementasikan prinsip *Last In First Out* (LIFO) di atas memori linear. Struktur ini banyak digunakan dalam pemrosesan rekursi, evaluasi ekspresi, dan *call stack* pada eksekusi program.

1.3 Abstraksi: Menjembatani Dunia Nyata dan Dunia Komputasional

Definisi: Abstraksi adalah proses transformasi suatu persoalan yang bersumber dari dunia nyata menjadi sebuah model matematis atau komputasional yang dapat dianalisis dan diselesaikan secara formal.

Kemampuan melakukan abstraksi merupakan keterampilan terpenting seorang *problem solver*. Semakin banyak pengalaman dalam menyelesaikan berbagai jenis soal, semakin baik kemampuan seseorang dalam melakukan abstraksi. Konsep matematika diskrit seperti kombinatorika, teori graf, dan teori bilangan sering menjadi perangkat utama dalam proses abstraksi.

Langkah-langkah Melakukan Abstraksi:

1. **Observasi Kreatif:** Amati permasalahan dari berbagai sudut pandang untuk mengidentifikasi pola atau regularitas yang tersembunyi.
2. **Pembuatan Tabel:** Gunakan tabel untuk menyusun data secara sistematis, sehingga relasi antar variabel menjadi lebih mudah dikenali.
3. **Pemodelan Matematis:** Rumuskan permasalahan dalam bentuk ekspresi atau persamaan matematis yang dapat dioperasikan secara komputasional.

1.4 Prinsip *Worst Case Scenario*

Seorang pemrogram yang andal senantiasa merancang solusinya berdasarkan kondisi masukan yang paling berat. Prinsip ini selaras dengan analisis formal algoritma yang menjadikan *worst case* sebagai tolok ukur utama. Dengan mempersiapkan solusi untuk skenario terburuk, kita memperoleh jaminan bahwa program akan berfungsi dengan benar dan efisien di semua kondisi, bukan hanya pada masukan yang menguntungkan.

Pembahasan Kasus: SPOJ CBANK — *Charu and Coin Distribution*

2.1 Deskripsi Permasalahan

Charu ingin menyetorkan tepat N rupee ke bank. Ia memiliki koin dengan empat denominasi: $\{0.25, 0.50, 1.00, 2.00\}$ rupee, masing-masing tersedia sebanyak $4N$ keping. Pertanyaannya: ada berapa banyak cara berbeda bagi Charu untuk menyusun pembayaran senilai tepat N rupee?

Batasan Masukan:

- $T \leq 10.000$ (jumlah kasus uji)
- $N \leq 10^9$
- Keluaran: jawaban modulo $10^9 + 7$

2.2 Langkah 1: Memahami Logika Soal

Sebelum merancang algoritma, pemahaman mendalam terhadap permasalahan adalah syarat mutlak. Dua kesalahan umum yang sering terjadi adalah langsung menulis kode tanpa memahami soal secara tuntas, serta mengabaikan batasan masukan yang dapat mempengaruhi pilihan algoritma.

Contoh untuk $N = 1$:

Kombinasi Koin	Total
4×0.25	1.00
$1 \times 0.50 + 2 \times 0.25$	1.00
2×0.50	1.00
1×1.00	1.00

Total: 4 cara. Catatan penting: urutan penyusunan koin tidak diperhitungkan, ini adalah permasalahan *combination*, bukan *permutation*.

2.3 Langkah 2: Abstraksi dan Penyederhanaan

Untuk memudahkan perhitungan bilangan bulat, seluruh nilai denominasi dikalikan dengan faktor 4:

$$0.25 \rightarrow 1, \quad 0.50 \rightarrow 2, \quad 1.00 \rightarrow 4, \quad 2.00 \rightarrow 8$$

Targetnya pun berubah dari N menjadi $4N$. Permasalahan kini setara dengan: mencari semua solusi bilangan bulat non-negatif (a, b, c, d) yang memenuhi:

$$8a + 4b + 2c + d = 4N$$

Dengan kata lain, persoalan distribusi koin telah diubah menjadi persoalan kombinatorial murni yang dapat diselesaikan secara matematis.

2.4 Langkah 3: Menemukan Pola melalui Observasi

Dengan menghitung nilai fungsi untuk beberapa nilai N secara manual, diperoleh deret berikut:

N	$f(N)$
1	4
2	10
3	20
4	35
5	56
6	84

Dengan menerapkan metode *finite difference* (selisih terbatas), diperoleh tabel berjenjang:

4	10	20	35	56	84
	6	10	15	21	28
		4	5	6	7
			1	1	1

Karena selisih terbatas tingkat ketiga bernilai konstan ($= 1$), dapat disimpulkan bahwa $f(N)$ merupakan **polinomial derajat 3**.

2.5 Langkah 4: Derivasi Model Matematika dengan *Newton's Forward Difference Formula*

Dengan memanfaatkan *Newton's Forward Difference Formula*, bentuk umum C_n dapat dituliskan sebagai:

$$C_n = 4 \binom{N-1}{0} + 6 \binom{N-1}{1} + 4 \binom{N-1}{2} + \binom{N-1}{3}$$

Koefisien 4, 6, 4, 1 berasal dari nilai-nilai awal pada tabel *finite difference*. Bentuk ini kemudian disederhanakan dengan menguraikan setiap suku kombinatorial:

$$\begin{aligned} \binom{N-1}{0} &= 1 \implies 4 \binom{N-1}{0} = 4 \\ \binom{N-1}{1} &= N-1 \implies 6 \binom{N-1}{1} = 6(N-1) \\ \binom{N-1}{2} &= \frac{(N-1)(N-2)}{2} \implies 4 \binom{N-1}{2} = 2(N-1)(N-2) \\ \binom{N-1}{3} &= \frac{(N-1)(N-2)(N-3)}{6} \end{aligned}$$

Sehingga persamaan akhirnya menjadi:

$$C_n = \left(4 + 6(N-1) + 2(N-1)(N-2) + \frac{(N-1)(N-2)(N-3)}{6} \right) \bmod (10^9 + 7)$$

Verifikasi:

- $N = 1$: $C_1 = 4 + 0 + 0 + 0 = 4 \checkmark$
- $N = 2$: $C_2 = 4 + 6 + 0 + 0 = 10 \checkmark$
- $N = 3$: $C_3 = 4 + 12 + 4 + 0 = 20 \checkmark$

2.6 Langkah 5: Penanganan Pembagian dalam Aritmetika Modular

Permasalahan muncul pada suku $\frac{(N-1)(N-2)(N-3)}{6}$, karena dalam aritmetika modular, pembagian langsung tidak dapat dilakukan. Sebagai gantinya, digunakan konsep **modular inverse**.

Modular Inverse dengan Teorema Kecil Fermat: Karena modulus $M = 10^9 + 7$ adalah bilangan prima, berlaku:

$$a^{-1} \equiv a^{M-2} \pmod{M}$$

Sehingga:

$$6^{-1} \equiv 6^{10^9+5} \pmod{10^9+7}$$

Nilai pangkat ini dihitung secara efisien menggunakan algoritma *modular exponentiation* (binary exponentiation), dengan kompleksitas $O(\log M)$.

2.7 Langkah 6: Implementasi — Solusi Optimal dengan Operasi Bitwise

Solusi berikut merupakan implementasi lengkap yang menggabungkan seluruh komponen polinomial sekaligus, tanpa perlu memisahkan kasus genap dan ganjil.

Listing 1: SPOJ CBANK — Implementasi Optimal

```

1 #include <cstdio>
2 using namespace std;
3 typedef long long ll;
4 ll tc, n, ans, MOD = 1000000007;
5
6 ll modexp(ll b, ll e, ll m) {
7     ll r = 1;
8     while (e > 0) {
9         if ((e & 1) == 1) {           // cek bit terkecil dengan bitwise
10             r = (r * b) % m;         AND
11         }
12         e >>= 1;                     // geser bit ke kanan (dibagi 2)
13         b = (b * b) % m;
14     }
15     return (ll)r;
16 }
17
18 int main() {
19     scanf("%lld", &tc);
20     while (tc--) {

```

```

21     scanf("%lld", &n);
22     ans = (4 + (n - 1) * 6) % MOD; //
        Suku konstan + linear
23     ll tmp = (((n - 1) * (n - 2)) % MOD) * 2) % MOD;
24     ans = ans + tmp;
        // Tambah suku kuadratik
25     ll inv6 = modexp(6, MOD - 2, MOD); //
        Hitung 6^{-1} mod M
26     tmp = (((n-1) * (n-2) % MOD) * (n-3) % MOD) * inv6 % MOD;
27     ans = ans + tmp;
        // Tambah suku kubik
28     printf("%lld\n", ans % MOD);
29 }
30 return 0;
31 }

```

Penjelasan Komponen Kode:

1. **Fungsi modexp:** Mengimplementasikan *binary exponentiation* secara iteratif. Operasi $e \& 1$ memeriksa bit paling rendah menggunakan bitwise AND, cara ini jauh lebih cepat dibandingkan operasi modulo biasa ($e \% 2$). Kompleksitas: $O(\log e)$.
2. **Tiga Suku Polinomial:** Rumus akhir diekspresikan sebagai akumulasi tiga komponen:
 - Suku konstan + linear: $4 + 6(N - 1)$
 - Suku kuadratik: $2(N - 1)(N - 2)$
 - Suku kubik: $\frac{(N - 1)(N - 2)(N - 3)}{6}$

Bentuk ini merupakan ekspansi eksplisit dari $\binom{N-1}{0} + \binom{N-1}{1} + \binom{N-1}{2} + \binom{N-1}{3}$.

3. **Modular Inverse:** Pembagian dengan 6 pada suku kubik diselesaikan menggunakan Teorema Kecil Fermat. Karena $M = 10^9 + 7$ adalah prima, maka $6^{-1} \equiv 6^{M-2} \pmod{M}$.

Analisis Kompleksitas Akhir:

- **Time Complexity:** $O(\log \text{MOD})$ per kasus uji. Karena MOD adalah konstanta, secara efektif ini setara dengan $O(1)$ per *query*.
- **Space Complexity:** $O(1)$, hanya menggunakan sejumlah variabel skalar.

Kompleksitas ini sangat efisien untuk batasan $T \leq 10.000$ dan $N \leq 10^9$.

Kesimpulan

Dari keseluruhan materi yang telah dipelajari, beberapa butir penting dapat dirangkum sebagai berikut:

1. **Fondasi Konseptual:** Ketiga elemen utama algoritma, program, dan struktur data merupakan pilar yang saling menopang dalam ilmu komputer. Pernyataan Niklaus Wirth, *Program = Algoritma + Struktur Data*, merangkum relasi esensial ketiganya dengan sangat tepat.
2. **Correctness dan Efficiency:** Dua prinsip utama yang harus dipenuhi oleh setiap algoritma adalah kebenaran (*correctness*) dan efisiensi (*efficiency*). Kebenaran dapat dibuktikan secara formal melalui metode seperti *loop invariant* dan induksi matematika, sementara efisiensi diukur melalui analisis kompleksitas waktu dan ruang.
3. **Abstraksi sebagai Keterampilan Inti:** Kemampuan mengubah permasalahan dunia nyata ke dalam model matematis yang tepat merupakan kompetensi paling mendasar seorang *problem solver*. Kualitas abstraksi akan terus berkembang seiring bertambahnya pengalaman dan eksplorasi berbagai jenis permasalahan.
4. **Metodologi Pemecahan Masalah yang Sistematis:** Studi kasus CBANK mendemonstrasikan alur berpikir terstruktur yang dapat diterapkan secara universal:

Pahami Soal → Abstraksi → Observasi Pola → Pemodelan Matematis → Implementasi

5. **Pentingnya *Worst Case Thinking*:** Perancangan solusi yang berfokus pada skenario terburuk bukan sekadar praktik yang baik, ini adalah standar yang harus dipenuhi dalam pengembangan sistem yang andal.
6. **Penguasaan Teknik Optimasi:** Penggunaan *binary exponentiation* dengan operasi bitwise, serta penerapan modular inverse melalui Teorema Kecil Fermat, menunjukkan bagaimana pemahaman matematika dan arsitektur komputer dapat dipadukan untuk menghasilkan solusi yang sangat efisien, bahkan untuk masukan berskala miliaran.

Bukti *Accepted*

[GilbranNRP134: submissions](#)
[Charu and Coin Distribution](#)

ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35574076	 2026-03-04 06:04:31	Charu and Coin Distribution	accepted edit delete it	0.01	5.2M	CPP14

Gambar 1: Screenshot bukti Accepted pada SPOJ problem 13626 — Charu and Coin Distribution